# SBA301 – SLOT 22

## CHAPTER 15 – IMPLEMENT CRUD APPLICATION WITH RESTFUL WEB SERVICE AND REACTJS

### PART A: CRUD BACKEND • SPRING BOOT REST API • SPRING DATA JPA • SQL SERVER

**TƯ DUY TRUNG TÂM:** CRUD backend không phải là "viết 4 endpoint". Sinh viên phải nhìn được use case → HTTP contract → Controller → Service → Repository → JPA/Hibernate → SQL Server, đồng thời kiểm chứng business rule, transaction boundary, validation, status code và hậu điều kiện dữ liệu.

| Thông tin | Giá trị |
|---|---|
| Môn học | SBA301 – Integrate Single Page Application with Spring Boot |
| Slot | 22 |
| Chủ đề | Chapter 15 Part A – Implement CRUD Application: CRUD Backend |
| CLO chính | CLO4 – Apply RESTful Web Service + JPA in full-stack application sequence |
| Case study xuyên suốt | demo_slot22 / FUNewsManagementSystem – Category + NewsArticle |
| Môi trường | JDK 21 • Spring Boot 3.x course baseline • Maven • Spring Web • Spring Data JPA • SQL Server • Postman/Swagger • Git |
| Thời lượng gợi ý | 135 phút trên lớp + bài tập/tự học |
| Learning flow | Prepare → Understand → Trace → Implement → Break → Diagnose → Verify → Transfer |

**SCOPE LOCK:** Slot 22 chỉ tập trung backend. ReactJS, Axios và UI CRUD thuộc Part B/slot tiếp theo. Không dạy lại sâu JPA căn bản của các slot trước; thay vào đó dùng JPA để xây một backend CRUD có contract, rule, test và evidence. Security/JWT chưa phải trọng tâm.

## CÁCH SỬ DỤNG GIÁO TRÌNH

| Pha | Sinh viên làm gì? | Evidence |
|---|---|---|
| Prepare | Chạy baseline; xác nhận JDK/Maven/SQL Server; đọc schema FUNews. | Baseline run + pre-flight checklist |
| Understand | Phân biệt CRUD use case, HTTP contract, layer boundary, entity state. | CRUD contract map |
| Trace | Trace một POST/PUT từ HTTP đến SQL và quay lại response. | Execution trace |
| Implement | Hoàn thiện Category/NewsArticle CRUD backend. | Source + Postman/Swagger evidence |
| Break | Cố ý tạo duplicate/missing FK/invalid body/wrong method. | Failure evidence |
| Diagnose | Đọc status, stack trace, Hibernate SQL, constraint error. | Debug note |
| Verify | Kiểm chứng API + SQL hậu điều kiện + negative tests. | Test matrix + DB evidence |
| Transfer | Áp dụng pattern sang Account/Tag/Product/Order module. | Design worksheet |

**AI RULE:** AI là learning aid. Mọi claim về execution flow phải được kiểm bằng source/debugger/log/SQL; mọi claim về version/xu hướng phải đối chiếu official documentation. Không dùng AI output thay evidence.

## 0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

### 0.1. Kiến thức cần ôn lại

- HTTP methods GET/POST/PUT/DELETE, request/response, JSON, status code 200/201/204/400/404/409/500.
- @RestController, @RequestMapping, @PathVariable, @RequestParam, @RequestBody, ResponseEntity.
- JPA entity mapping: @Entity, @Table, @Id, @GeneratedValue, @Column; relationship và foreign key.
- JpaRepository<T, ID>, findAll/findById/save/delete; derived query cơ bản.
- Layered architecture: Controller → Service → Repository; dependency injection; transaction boundary.
- Slot 20: frontend/backend là hai boundary khác nhau; Slot 22 chưa viết React nhưng phải thiết kế API sẵn cho client.

### 0.2. Công cụ và baseline

| Thành phần | Yêu cầu | Kiểm tra nhanh |
|---|---|---|

| | | |
|---|---|---|
| JDK | 21 | java -version |
| Maven | 3.6.3+ / Maven wrapper | mvn -v hoặc mvnw -v |
| IDE | IntelliJ IDEA | project import/build xanh |
| SQL Server | Instance hoạt động; schema/sample data sẵn sàng | SSMS SELECT smoke test |
| Spring dependencies | Spring Web, Spring Data JPA, SQL Server Driver, Validation, Test | Maven reload |
| Postman/Swagger | Gửi request và xem status/body | GET smoke test |
| Git | Baseline sạch | git status / git log |

## 0.3. Pre-class 3–2–1 Learning Note

| 3 điều đã biết | 2 điều còn mơ hồ | 1 câu hỏi muốn kiểm chứng |
|---|---|---|
| save() có thể tạo/cập nhật; Controller nhận HTTP; Repository làm persistence | PUT khác POST ở semantics nào? Khi nào trả 404/409? | Một POST NewsArticle đi qua method nào và SQL nào? |

## 0.4. Pre-flight checklist

- SQL Server chạy và có quyền tạo/đọc DB.
- Project compile trước khi sửa.
- application.properties dùng đúng datasource URL/user/password.
- Có Postman collection hoặc Swagger UI.
- Bật Hibernate SQL log ở mức phù hợp cho trace.
- Git checkpoint gợi ý: s22-before-crud-backend.

```
mvn -q -DskipTests compile
mvn test
mvn spring-boot:run
```

**BASELINE GATE:** PASS khi application start, kết nối DB thành công, không có entity mapping error, và GET health/smoke endpoint trả response.

# I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

## 1.1. Chapter 15 Part A trong course sequence

| Nền tảng trước Slot 22 | Slot 22 nâng lên | Slot sau sử dụng |
|---|---|---|
| REST + JPA riêng lẻ | Backend CRUD hoàn chỉnh theo resource/use case | React/Axios gọi API CRUD |
| Entity/Repository/Service | Business rule + HTTP contract + persistence verification | Full-stack state + UI workflow |
| Manual endpoint demo | Positive/negative tests + error semantics | Frontend error handling |

Chapter 15 mô tả server side bằng Spring REST API + Spring Data JPA, database MS SQL Server; Part 1 gồm setup project, entity, data access, business logic, REST controller và chạy backend. Đây là phạm vi cốt lõi của Slot 22.

## 1.2. Mục tiêu học tập

1. Giải thích CRUD theo use case và HTTP semantics, không đồng nhất CRUD database với REST API.
2. Thiết kế project Spring Boot backend theo Controller → Service → Repository → SQL Server.
3. Map entity và relationships từ schema thực; phân biệt API model và persistence model ở mức awareness.
4. Thực hiện Create/Read/Update/Delete/Search bằng Spring Data JPA và REST endpoints.
5. Áp dụng validation và business constraints trước khi persistence.
6. Dùng status code hợp lý: 200, 201, 204, 400, 404, 409; hiểu 500 không phải "lỗi mặc định".
7. Trace POST/PUT/DELETE tới SQL và verify hậu điều kiện trong database.
8. Phân tích delete rule, FK constraint, transaction boundary và concurrent update ở mức nền tảng.
9. Viết test matrix và evidence cho backend trước khi frontend tích hợp.
10. Transfer pattern sang Account, Tag hoặc module sản phẩm thực tế.

## 1.3. Mental model: CRUD backend là pipeline có contract

```
HTTP Client
  -> DispatcherServlet / HandlerMapping
  -> REST Controller (HTTP boundary)
  -> Service (use case + business rule + transaction)
  -> Repository (persistence abstraction)
  -> JPA/Hibernate / EntityManager
  -> JDBC Driver
  -> SQL Server
  <- row/entity/result
  <- JSON serialization + status code
HTTP Response
```

**COURSE-READY RULE:** API "chạy được" nhưng sai status, bỏ business rule, hoặc persistence state sai vẫn là backend chưa đúng.

# II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

## 2.1. CRUD database vs REST semantics

| Use case | HTTP | Repository thường dùng | Success | Failure điển hình |
|---|---|---|---|---|
| Create | POST /api/categories | save(newEntity) | 201 Created | 400 invalid; 409 conflict |
| Read list | GET /api/categories | findAll/Page | 200 OK | — |
| Read detail | GET /api/categories/{id} | findById | 200 OK | 404 Not Found |
| Update | PUT /api/categories/{id} | findById + save | 200/204 | 400/404/409 |
| Delete | DELETE /api/categories/{id} | delete/deleteById | 204 | 404/409 |

## 2.2. Layer boundary

| Layer | Nên làm | Không nên làm |
|---|---|---|
| Controller | Route, bind, validate trigger, status/headers/body, call service | Business rule dài; repository query trực tiếp |
| Service | Use case, rule, orchestration, @Transactional | Biết URL/HTTP status chi tiết |
| Repository | CRUD/query persistence abstraction | HTTP concern; workflow nghiệp vụ |
| Entity | Persistence mapping + invariant đơn giản | Trở thành API contract cho mọi client mà không cân nhắc coupling |

## 2.3. save() không đồng nghĩa "insert"

Trong Spring Data JPA, save(entity) có thể dẫn tới persist hoặc merge tùy entity state/identifier. Vì vậy service create/update phải làm rõ pre-condition thay vì gọi save() mù quáng.

| Tình huống | Sai lầm | Course-ready direction |
|---|---|---|
| Create có ID do client gửi | save() có thể thành update/merge ngoài ý muốn | Quy định ID generation và validate request |
| Update ID không tồn tại | save() tạo row mới | findById trước, nếu thiếu → 404 |
| Detached entity | Copy toàn body trực tiếp | Load managed entity rồi cập nhật field theo contract |

## 2.4. Validation vs business validation

| Loại | Ví dụ | Đặt ở đâu |
|---|---|---|
| Syntactic/Bean Validation | @NotBlank name; @Size; email format | Request DTO / boundary |
| Business validation | CategoryName unique; cannot delete used category | Service/use case |
| DB integrity | FK, unique constraint, NOT NULL | SQL Server as final integrity gate |

## 2.5. Delete là use case, không phải chỉ deleteById()

**CASE STUDY:** Assignment 02 quy định: Account chỉ được delete khi chưa tạo NewsArticle; Category chỉ được delete khi chưa được NewsArticle tham chiếu. Đây là ví dụ business rule phải kiểm tra ở Service, không giao hoàn toàn cho DB exception.

## 2.6. Transaction boundary

- Read-only use case có thể dùng @Transactional(readOnly = true).
- Create/update/delete nhiều bước nên đặt transaction ở Service.
- Không mở transaction ở Controller.
- Không giả định transaction local giải quyết được external systems; phần này chỉ awareness.

## 2.7. Search, paging, sorting – tránh findAll vô hạn

```
Page<Category> findByCategoryNameContainingIgnoreCase(String keyword, Pageable pageable);
```

Ở course core có thể dùng List cho dataset nhỏ; production direction nên dùng Pageable/Page hoặc Slice với giới hạn page size.

## 2.8. DTO và Entity – mức nhận thức cần có

| Entity trực tiếp | DTO request/response |
|---|---|
| Nhanh cho lab nhỏ | Tách API contract khỏi database model |
| Dễ lộ relationship/lazy field | Kiểm soát field và validation |
| Coupling cao | Dễ version/refactor hơn |

# III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

## 3.1. Case study – FUNewsManagementSystem

| Resource | Fields/relationship chính | CRUD concern |
|---|---|---|
| Category | self parent; IsActive; 1–N NewsArticle | CRUD/search; delete constraint |
| NewsArticle | belongs Category; CreatedBy SystemAccount; N–N Tag | CRUD; status; timestamps; relationship |
| SystemAccount | role, email, password; creates NewsArticle | CRUD + business delete rule |
| Tag | N–N NewsArticle | CRUD/reuse |
| NewsTag | join table | association management |

## 3.2. Từ business rule tới API contract

| Use case | Endpoint | Rule | Expected |
|---|---|---|---|
| List categories | GET /api/categories?keyword=&page= | filter + paging | 200 |
| Create category | POST /api/categories | name valid/unique | 201 |
| Update category | PUT /api/categories/{id} | id exists | 200/404 |
| Delete category | DELETE /api/categories/{id} | must not be referenced | 204/409 |
| Create article | POST /api/news | category/account exist | 201/400/404 |
| Delete account | DELETE /api/accounts/{id} | cannot delete if created news | 204/409 |

## 3.3. Product thực tế tương tự

| Domain | CRUD resource | Rule tương tự |
|---|---|---|
| E-commerce | Category/Product | Không xóa category đang có product |
| HR | Department/Employee | Không xóa department còn employee |
| Booking | RoomType/Room | Không xóa loại phòng đang dùng |
| Content | Author/Post | Xử lý author đã có content |

## 3.4. Beginner backend vs production backend

| Vấn đề | Beginner | Production direction |
|---|---|---|
| Input | Entity trực tiếp | Request DTO + validation |
| Error | throw → 500 | standardized error / ProblemDetail |
| List | findAll | paging/filter/sort |
| Delete | deleteById | business rule + 404/409 |
| Concurrency | last write wins | optimistic locking/version when needed |
| Schema | ddl-auto=update | migration tool + controlled schema |
| Secrets | password trong properties | environment/secret store |
| Tests | Postman only | unit + slice + integration tests |

# IV. NỘI DUNG CHI TIẾT – NỀN TẢNG LÝ THUYẾT, KIẾN TRÚC VÀ HỆ SINH THÁI KIẾN THỨC

| MỤC TIÊU CỦA PHẦN IV |
|---|
| Phần IV là knowledge base của Slot 22. Sinh viên phải hiểu khái niệm, semantics, lifecycle, trách nhiệm kiến trúc và quan hệ giữa Spring MVC – Service – Spring Data JPA – Hibernate – SQL Server trước khi bước sang phần VI để thao tác code. Phần này KHÔNG phải checklist cài đặt và không thay thế tutorial step-by-step. |

## 4.1. Bức tranh tổng thể: CRUD backend nằm ở đâu trong full-stack SPA?

Trong Chapter 15, hệ thống hoàn chỉnh gồm React client, Spring Boot REST API, Spring Data JPA và SQL Server. Part A của Slot 22 khóa phạm vi ở backend: biến HTTP request thành use case nghiệp vụ, duy trì trạng thái dữ liệu đúng trong database và trả HTTP response có contract rõ ràng. React chỉ là consumer ở Part B, không phải trọng tâm của Slot này.

```
Client (Postman / React later)
  -> HTTP Request
  -> Spring MVC / DispatcherServlet
  -> REST Controller
  -> Service / Business Use Case
  -> Repository (Spring Data JPA)
  -> Hibernate / EntityManager / JDBC
  -> SQL Server
  <- result / exception
  <- JSON + HTTP status
```

| Layer | Trách nhiệm chính | Không nên chứa |
|---|---|---|
| Controller | HTTP boundary: route, binding, validation trigger, status/header/body | SQL/query chi tiết; transaction workflow; business rule lớn |
| Service | Use case, business rules, orchestration, transaction boundary | URL, HTTP status, JSON concerns |
| Repository | Persistence abstraction, CRUD/query contract | HTTP concerns; nghiệp vụ nhiều bước |
| Database | Constraint, referential integrity, persistence durability | Business workflow ở tầng ứng dụng |

## 4.2. CRUD không chỉ là bốn nút Create – Read – Update – Delete

CRUD là mô hình thao tác dữ liệu nền tảng, nhưng trong REST backend mỗi operation phải được hiểu đồng thời ở ba lớp: HTTP semantics, application use case và database state transition.

| CRUD | HTTP thường dùng | Ý nghĩa application | Hậu điều kiện cần kiểm chứng |
|---|---|---|---|
| Create | POST | Tạo resource mới từ representation hợp lệ | Có bản ghi mới; ID/business key hợp lệ; FK đúng |
| Read | GET | Đọc resource/list/filter mà không mutate state | DB không thay đổi; representation đúng |
| Update | PUT/PATCH | Thay đổi resource theo contract | Chỉ field cho phép thay đổi; rule/constraint vẫn đúng |
| Delete | DELETE | Loại bỏ hoặc chuyển trạng thái resource | Không còn truy xuất như active; quan hệ không bị phá |
| **MENTAL MODEL** | | | |
| Một API “trả 200” chưa chứng minh CRUD đúng. Phải kiểm chứng cả HTTP contract và database post-condition. | | | |

## 4.3. REST resource, URI và HTTP method semantics

Resource-oriented design dùng danh từ cho URI và dùng HTTP method để biểu đạt hành động. Đây là nền tảng để client, test và tài liệu OpenAPI hiểu API một cách nhất quán.

| Mục đích | Khuyến nghị | Anti-pattern |
|---|---|---|
| List categories | GET /api/categories | GET /api/getAllCategories |

| | | |
|---|---|---|
| Read one | GET /api/categories/{id} | GET /api/getCategory?id=... |
| Create | POST /api/categories | POST /api/createCategory |
| Update | PUT /api/categories/{id} | POST /api/updateCategory |
| Delete | DELETE /api/categories/{id} | GET /api/deleteCategory/{id} |

### 4.3.1. Safe và idempotent

GET được xem là safe vì không được thiết kế để thay đổi business state. PUT và DELETE có tính idempotent theo semantics: lặp lại cùng request hợp lệ không nên tạo thêm hiệu ứng ngoài dự kiến. POST thường không idempotent vì mỗi lần gọi có thể tạo resource mới.

## 4.4. HTTP request/response contract

Backend không chỉ trả object Java. Nó cung cấp một HTTP contract gồm method, URI, headers, body, status code và error shape. Contract phải đủ ổn định để frontend và automated test phụ thuộc vào.

```
HTTP Request = method + URI + headers + optional body
HTTP Response = status + headers + optional body
```

| Case | Status gợi ý | Body |
|---|---|---|
| GET success | 200 OK | resource/list/page |
| POST success | 201 Created | created representation; có thể thêm Location |
| PUT success | 200 OK hoặc 204 No Content | updated representation hoặc empty |
| DELETE success | 204 No Content | empty |
| Invalid input | 400 Bad Request | standardized error |
| Missing resource | 404 Not Found | standardized error |
| Conflict/business constraint | 409 Conflict | standardized error |
| Unexpected server failure | 500 Internal Server Error | không rò stack trace |

## 4.5. Spring Boot startup lifecycle và dependency graph

Hiểu startup flow giúp sinh viên biết vì sao Controller/Service/Repository có thể được inject mà không tự new object. Spring Boot khởi tạo ApplicationContext, scan component, auto-configure MVC/DataSource/JPA, tạo Spring Data repository proxy rồi resolve dependency graph.

```
main()
 -> SpringApplication.run(...)
 -> create/refresh ApplicationContext
 -> component scanning
 -> auto-configuration (MVC, DataSource, JPA, transaction manager)
 -> entity metadata + EntityManagerFactory
 -> Spring Data creates repository proxies
 -> create Service beans
 -> create Controller beans
 -> embedded server + DispatcherServlet ready
```

### 4.5.1. Inversion of Control và Dependency Injection

IoC nghĩa là application không tự quản lý toàn bộ vòng đời dependency. Spring container tạo bean và inject dependency. Constructor injection được ưu tiên vì dependency hiển nhiên, field có thể final và unit test dễ hơn.

```
@Service
public class CategoryServiceImpl {
    private final CategoryRepository repository;

    public CategoryServiceImpl(CategoryRepository repository) {
        this.repository = repository;
    }
}
```

## 4.6. Domain model và JPA Entity

Entity là object có identity được JPA quản lý và ánh xạ tới relational schema. Entity không chỉ là "class có @Entity"; mapping phải phản ánh đúng primary key, column, nullability, relationship và lifecycle của dữ liệu.

| Annotation | Ý nghĩa | Điểm cần hiểu |
|---|---|---|
| @Entity | Đánh dấu persistent entity | Class tham gia JPA metadata |
| @Table | Chỉ định table/schema | Hữu ích khi tên Java khác tên DB |
| @Id | Primary key | Identity của entity |
| @GeneratedValue | Chiến lược sinh ID | IDENTITY thường map SQL Server identity |
| @Column | Tên/constraint column | Không thay thế toàn bộ DB constraint |
| @ManyToOne | N–1 relationship | Owning side thường chứa FK |
| @OneToMany | 1–N relationship | mappedBy xác định inverse side |

### 4.6.1. Java naming và physical database naming

Tên property Java và tên cột SQL không nhất thiết giống nhau. Derived query của Spring Data dùng tên PROPERTY Java, còn @Column dùng tên COLUMN database. Đây là nguồn lỗi phổ biến khi sinh viên nhầm `categoryName` với `CategoryName`.

| CASE STUDY NOTE |
|---|
| Database Assignment 02 có cột `CategoryDesciption` (sai chính tả "Description"). Nếu dùng schema có sẵn, mapping phải bám tên cột thật hoặc phải migrate schema có chủ đích; không được âm thầm giả định tên đúng. |

## 4.7. Persistence Context và lifecycle của Entity

Hibernate/JPA làm việc thông qua persistence context. Hiểu trạng thái entity giúp giải thích vì sao một object được update, khi nào SQL chạy và vì sao thao tác trên detached object có thể khác mong đợi.

| Trạng thái | Ý nghĩa | Ví dụ |
|---|---|---|
| Transient/New | Object chưa được persistence context quản lý | new Category(...) |
| Managed/Persistent | Entity đang được EntityManager theo dõi | findById trong transaction |
| Detached | Từng managed nhưng không còn trong context | Entity ra ngoài transaction/context |
| Removed | Được đánh dấu xóa | repository.delete(entity) |

### 4.7.1. Dirty checking

Trong transaction, nếu entity đang managed và field được thay đổi, Hibernate có thể phát hiện thay đổi (dirty checking) và phát SQL UPDATE khi flush/commit. Vì vậy update không phải lúc nào cũng cần gọi save() sau mọi setter; tuy nhiên ở course level có thể dùng save() để flow dễ quan sát, miễn sinh viên hiểu cơ chế bên dưới.

### 4.7.2. Flush và commit

`save()` không đồng nghĩa "SQL đã commit ngay". SQL có thể được phát khi flush; transaction commit mới xác lập transaction thành công. Khi debug, cần phân biệt method return, flush và database commit.

## 4.8. Spring Data JPA Repository abstraction

`JpaRepository<T, ID>` cung cấp CRUD, paging/sorting và tích hợp JPA. Spring tạo implementation proxy runtime nên sinh viên định nghĩa interface thay vì viết DAO boilerplate cho các thao tác chuẩn.

```
public interface CategoryRepository extends JpaRepository<Category, Short> {
    Optional<Category> findByCategoryNameIgnoreCase(String categoryName);
    boolean existsByCategoryNameIgnoreCase(String categoryName);
    Page<Category> findByCategoryNameContainingIgnoreCase(String keyword, Pageable pageable);
}
```

### 4.8.1. Derived query

Spring Data phân tích method name theo property của entity. Ví dụ `findByCategoryNameContainingIgnoreCase` dựa trên property `categoryName`, không dựa trên tên cột `CategoryName`. Method name sai property sẽ gây lỗi startup hoặc query creation.

### 4.8.2. Optional

`Optional<T>` biểu đạt khả năng không tìm thấy resource rõ hơn việc trả `null`. Service quyết định chuyển "missing entity" thành business/application outcome; Controller mới map outcome đó sang HTTP 404.

## 4.9. Service layer và transaction boundary

Service là nơi mô hình hóa use case. Một use case có thể cần nhiều repository call, validation nghiệp vụ và transaction. Vì vậy transaction boundary thường đặt ở Service thay vì Controller.

| Service responsibility | Ví dụ FUNewsManagement |
|---|---|
| Business invariant | Không cho xóa Category đang được NewsArticle tham chiếu |
| Orchestration | find category → kiểm tra rule → update → save |
| Transaction | Nhiều bước thay đổi phải all-or-nothing |
| Repository coordination | CategoryRepository + NewsArticleRepository |
| Domain/application errors | NotFound, Conflict, validation outcome |

### 4.9.1. @Transactional

`@Transactional` định nghĩa transaction boundary declaratively. `readOnly=true` phù hợp cho read use case như một optimization hint và diễn đạt intent; write use case dùng transaction thông thường.

```
@Transactional(readOnly = true)
public Category getById(short id) { ... }

@Transactional
public Category update(short id, CategoryRequest request) { ... }
```

## 4.10. REST Controller: HTTP boundary, không phải business layer

Controller nhận request, chọn handler theo mapping, bind dữ liệu, trigger validation, gọi Service và tạo HTTP response. Controller mỏng giúp logic nghiệp vụ test độc lập khỏi servlet/web layer.

| Annotation | Vai trò |
|---|---|
| @RestController | Controller + serialize return value vào response body |
| @RequestMapping | Base path/criteria |
| @GetMapping | GET handler |
| @PostMapping | POST handler |
| @PutMapping | PUT handler |
| @DeleteMapping | DELETE handler |
| @PathVariable | Bind identity từ URI path |
| @RequestParam | Bind filter/page/sort option |
| @RequestBody | Deserialize JSON body thành Java object |
| @Valid | Trigger Bean Validation cho request object |

## 4.11. JSON serialization/deserialization lifecycle

Spring MVC dùng HttpMessageConverter; với JSON thường là Jackson. Request JSON được deserialize thành DTO/entity trước khi controller method chạy; response object được serialize thành JSON sau khi controller trả về.

```
JSON request
 -> HttpMessageConverter / Jackson
 -> Request DTO
 -> Controller
 -> Service
 -> Repository/JPA
 -> SQL Server
 <- entity/result
 <- Response DTO
 <- Jackson serialization
 <- JSON response
```

| **RELATIONSHIP WARNING** |
|---|
| JPA relationship đúng không đảm bảo JSON an toàn. Bidirectional relationship có thể gây recursion hoặc payload quá lớn. Production direction là dùng DTO/projection có chủ đích thay vì expose toàn bộ entity graph. |

## 4.12. Validation: ba lớp không được đánh đồng

| Lớp validation | Ví dụ | Công cụ |
|---|---|---|
| Syntactic/Input | name không rỗng, max 100 ký tự | Bean Validation: @NotBlank, @Size |
| Business | category name phải unique; không xóa khi đang được dùng | Service rule + repository check |
| Database integrity | NOT NULL, UNIQUE, FK | SQL Server constraint |

Input validation nên fail sớm trước khi đi sâu vào database. Business validation cần ở Service vì nó mô tả rule của use case. DB constraint là hàng rào integrity cuối cùng, không nên là nơi duy nhất phát hiện lỗi nghiệp vụ.

## 4.13. DTO, Entity và API contract

Course slide có thể trả Entity trực tiếp để đơn giản hóa, nhưng hệ thống production thường tách request/response DTO khỏi persistence entity. DTO giúp kiểm soát fields, tránh mass assignment, tránh lazy relationship leak và giảm coupling giữa DB schema với public API.

| Tiêu chí | Expose Entity | DTO |
|---|---|---|
| Đơn giản demo | Nhanh | Cần mapping |
| API stability | Couple vào entity | Tách contract |
| Sensitive fields | Dễ lộ nếu bất cẩn | Chỉ chọn field cần |
| Relationships | Dễ recursion/lazy issue | Chủ động shape |
| Validation theo use case | Khó tách create/update | Dễ tạo request model riêng |

## 4.14. Error handling và standardized error contract

Exception Java không nên tự động trở thành response ngẫu nhiên. Global exception handling giúp chuẩn hóa HTTP status và error body để frontend có thể xử lý nhất quán.

```
@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> notFound(ResourceNotFoundException ex) { ... }

    @ExceptionHandler(BusinessConflictException.class)
    ResponseEntity<ApiError> conflict(BusinessConflictException ex) { ... }
}
```

| Lỗi | Status | Ví dụ |
|---|---|---|
| Input invalid | 400 | blank category name |
| Not found | 404 | category id không tồn tại |
| Business conflict | 409 | xóa category đang có news article |
| Unsupported media type | 415 | text/plain thay vì JSON |
| Unexpected exception | 500 | bug/DB outage; không trả stack trace cho client |

## 4.15. Relationship và referential integrity trong FUNewsManagement

Assignment 02 có Category, NewsArticle, SystemAccount, Tag và NewsTag. Database script thiết lập FK giữa NewsArticle–Category, NewsArticle–SystemAccount và NewsTag–NewsArticle/Tag. CRUD backend phải tôn trọng các quan hệ này thay vì coi mỗi table là độc lập.

| Quan hệ | Ý nghĩa backend |
|---|---|
| Category 1 – N NewsArticle | Create/update article phải tham chiếu category hợp lệ |
| SystemAccount 1 – N NewsArticle | CreatedByID phải tồn tại và phản ánh creator |
| NewsArticle N – N Tag qua NewsTag | Create/update tags cần đồng bộ join rows |
| Category self-reference ParentCategoryID | Cần tránh parent invalid/cycle nếu business rule yêu cầu |
| **ASSIGNMENT RULE** | |
| Assignment yêu cầu không xóa Account/Category khi resource đã được NewsArticle sử dụng. Đây là business rule ở Service; không nên chỉ chờ SQL exception từ FK. | |

## 4.16. Delete semantics: hard delete, soft delete và business delete

DELETE không mặc định đồng nghĩa `DELETE FROM table`. Tùy domain có thể hard delete, soft delete (`IsActive=false`) hoặc reject bằng conflict. Course/assignment phải xác định rõ post-condition mong muốn.

| Kiểu | Khi dùng | Trade-off |
|---|---|---|
| Hard delete | Dữ liệu có thể xóa vĩnh viễn và không bị tham chiếu | Đơn giản nhưng mất lịch sử |
| Soft delete | Cần lịch sử/audit/khả năng phục hồi | Mọi query phải lọc trạng thái |
| Reject delete | Resource đang được tham chiếu hoặc rule cấm | Client cần 409/error rõ |

## 4.17. Search, filter, sort và pagination

CRUD hệ thống thực tế thường cần Search. `findAll()` không phải thiết kế cho collection lớn. Spring Data hỗ trợ Pageable/Sort; API nên có page/size/sort và giới hạn page size để tránh payload quá lớn.

```
GET /api/news-articles?keyword=AI&categoryId=1&page=0&size=20&sort=createdDate,desc
```

| Concern | Rule course-ready |
|---|---|
| page | 0-based hoặc 1-based phải thống nhất và document |
| size | Có default và max |
| sort | Chỉ allow field phù hợp |
| filter | Optional query parameter |
| search | Derived query/specification/query method theo mức bài học |

## 4.18. Update semantics: PUT, PATCH và field ownership

PUT thường biểu đạt replacement/update đầy đủ theo contract; PATCH biểu đạt partial update. Dù dùng method nào, server phải kiểm soát field nào client được phép thay đổi. Không copy mù mọi field từ request vào entity, đặc biệt ID, createdDate, createdBy, role hoặc các field hệ thống.

| MASS ASSIGNMENT WARNING |
|---|
| Không dùng `BeanUtils.copyProperties(request, entity)` một cách vô điều kiện cho model có field nhạy cảm. Mapping explicit làm rõ ownership của từng field. |

## 4.19. Concurrency, optimistic locking và lost update – awareness

Khi hai client cùng sửa một record, “last write wins” có thể làm mất thay đổi. Ở mức production awareness, JPA `@Version` hỗ trợ optimistic locking. Slot 22 không bắt buộc triển khai nhưng sinh viên cần nhận biết vì CRUD production không chỉ là gọi `save()`.

```
@Version
private Long version;
```

## 4.20. Idempotency và duplicate create – awareness

POST create có thể bị retry do mạng, double-click hoặc client timeout. Với resource có business key, unique constraint hoặc idempotency strategy giúp tránh duplicate. Đây là production direction; core Slot 22 chỉ cần hiểu vấn đề và kiểm soát duplicate theo rule đơn giản khi phù hợp.

## 4.21. Testing model: test đúng boundary

| Boundary | Tool/kiểu test | Mục tiêu |
|---|---|---|
| Controller | @WebMvcTest + MockMvc | mapping, JSON, status, validation, exception contract |
| Service | JUnit + Mockito | business rules, orchestration |
| Repository | @DataJpaTest | mapping, derived query, constraint assumptions |
| Full integration | @SpringBootTest + test database/Testcontainers khi phù hợp | wiring + multi-layer flow |
| Manual smoke | Postman/Swagger | exploration/demo; không thay regression suite |

## 4.22. Observability và debugging mental model

Debugging backend hiệu quả đi từ HTTP boundary vào trong: status/mapping → binding/validation → service rule → repository/query → generated SQL → database state. Log cần đủ context nhưng không chứa password/token/sensitive data.

```
Request fails
 |- 404 -> mapping/path/id?
 |- 400 -> JSON/type/validation?
 |- 405 -> method mismatch?
 |- 415 -> Content-Type?
 |- 409 -> business constraint?
 |- 500 -> stack trace -> Service? JPA? JDBC? Jackson?

Request succeeds but data wrong
 -> service rule -> repository query -> generated SQL -> DB state
```

## 4.23. Security boundary – chỉ awareness trong Part A

Assignment có role Admin/Staff nhưng authentication/authorization không phải trọng tâm Part A nếu syllabus chưa đến Security. Tuy vậy, thiết kế Service/Controller phải tránh hard-code role checks lẫn với persistence. Phần security sẽ được gắn vào cùng use case ở chapter/slot phù hợp.

## 4.24. Ba lifecycle sinh viên phải trace được

### 4.24.1. Startup lifecycle

```
main -> SpringApplication.run -> ApplicationContext -> DataSource/JPA -> repository proxy -> Service bean -> Controller bean -> server ready
```

### 4.24.2. CREATE request lifecycle

```
POST JSON -> DispatcherServlet -> HandlerMapping -> Jackson -> @Valid -> Controller -> Service @Transactional -> Repository.save -> Hibernate -> SQL INSERT -> commit -> response 201
```

### 4.24.3. UPDATE request lifecycle

```
PUT /{id} -> bind id/body -> validate -> Service -> findById -> business rules -> mutate managed entity -> dirty checking/save -> SQL UPDATE -> commit -> response
```

## 4.25. Anti-patterns cần nhận diện

| Anti-pattern | Vì sao sai/hạn chế | Hướng sửa |
|---|---|---|
| Controller gọi Repository trực tiếp | Trộn HTTP với persistence/business | Controller -> Service -> Repository |

| | | |
|---|---|---|
| 200 cho mọi outcome | Client không phân biệt success/error | Dùng status đúng semantics |
| Return null khi missing | Contract mơ hồ | Optional/exception -> 404 |
| findAll cho dữ liệu lớn | Latency/memory/payload | Pagination/filter |
| Expose entity graph vô hạn | Recursion/lazy/sensitive fields | DTO/projection |
| Catch Exception rồi trả message | Che lỗi, contract không nhất quán | Global exception handler |
| DB exception = business validation | UX kém, coupling | Check rule ở Service + DB constraint |
| Update copy mọi field | Mass assignment | Explicit mapping/DTO |
| Log password/token | Security incident | Sanitize structured logging |

### 4.26. Knowledge map – kết nối hệ sinh thái kiến thức

```
HTTP / REST semantics
  -> Spring MVC routing + binding + validation
  -> Application Service + transaction
  -> Spring Data Repository abstraction
  -> JPA Entity + Persistence Context
  -> Hibernate ORM + generated SQL
  -> SQL Server schema + constraints + indexes
  -> JSON contract / DTO / errors
  -> Tests + logs + debugging evidence
  -> React consumer ở Part B
```

Khi kết thúc Phần IV, sinh viên phải có thể giải thích bằng lời và sơ đồ: "một request CRUD đi qua những boundary nào, mỗi boundary chịu trách nhiệm gì, state thay đổi ở đâu, lỗi có thể phát sinh ở đâu và bằng chứng nào chứng minh kết quả đúng".

## V. KINH NGHIỆM THỰC TIỄN, TIPS

| Vấn đề | Triệu chứng | Cách xử lý |
|---|---|---|
| Sai DB name | Login OK nhưng table không thấy / invalid object | Đối chiếu JDBC databaseName + USE trong SQL script |
| ddl-auto=update trên DB shared | Schema tự thay đổi ngoài ý muốn | Lab có thể dùng; schema cố định dùng validate/migration |
| save() cho update missing | Tạo dữ liệu ngoài ý muốn | findById + 404 trước update |
| delete FK referenced | DataIntegrityViolationException/500 | Business check + 409; DB constraint vẫn giữ |
| Controller gọi repository | Code ngắn nhưng rule phân tán | Service boundary |
| Entity serialize vòng lặp | Huge JSON/StackOverflow | DTO/Jackson mapping deliberation |
| LazyInitializationException | Access relation ngoài context | DTO/query/fetch plan; không EAGER toàn bộ |
| N+1 | List phát nhiều SELECT | log SQL, projection/entity graph/query |
| findAll lớn | Latency/memory | Pageable/filter/max size |
| password trong git | Credential leak | env variable/secret store |

### 5.1. Debugging decision tree

```
Request fails
 |- 404 -> URL/mapping/id missing?
 |- 400 -> JSON/binding/validation/type?
 |- 405 -> wrong HTTP method?
 |- 409 -> business conflict / DB integrity?
 |- 500
    |- Controller/Service -> application exception
    |- Repository/Hibernate -> mapping/query/constraint
    |- SQL Server -> login/schema/table/FK

Request succeeds but data wrong
 -> service rule -> entity state -> repository -> generated SQL -> DB row
```

### 5.2. Production radar 2026 – chỉ để định hướng

| Chủ đề | Course baseline | Current/production direction |
|---|---|---|
| Spring Boot | 3.x + JDK 21 | Hệ sinh thái hiện tại đã có Boot 4.x; không ép migrate giữa kỳ |
| Spring Data JPA | JpaRepository CRUD/query | Current stable docs 4.1.x; projections/specifications/AOT là hướng mở rộng |
| Validation | @Valid + Bean Validation | Spring MVC hỗ trợ method validation; handle validation exceptions |

| | | |
|---|---|---|
| | | nhất quán |
| Error response | ResponseEntity/custom error | ProblemDetail / RFC 9457 direction |
| Schema | ddl-auto cho learning | Flyway/Liquibase + migration governance |
| Testing | Postman + JUnit | Testcontainers/integration CI khi phù hợp |

Production radar không thay course baseline. Sinh viên chỉ cần hiểu vấn đề mà công nghệ mới giải quyết và biết cách kiểm chứng bằng official docs.

# VI. HƯỚNG DẪN STEP BY STEP THỰC HIỆN BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG – IMPLEMENTATION WORKBOOK

| MỤC TIÊU CỦA PHẦN VI |
| --- |
| Phần VI là implementation workbook. Sinh viên thực hiện từng thao tác trên một Category CRUD backend độc lập, sau đó mở rộng sang NewsArticle. Mỗi bước đều có: Mục tiêu → Thao tác → Code/cấu hình → Lệnh thực thi → Kiểm chứng PASS/FAIL → Evidence cần lưu. |

## 6.1. Bài tập chủ đề – Xây dựng Category CRUD Backend từ máy đã cài công cụ

Case study: xây dựng module Category của FUNewsManagementSystem bằng Spring Boot 3.x course baseline, JDK 21, Maven, Spring Web, Spring Data JPA, Validation và SQL Server. API base path sử dụng `/api/categories`.

### 6.1.1. Bước 0 – Pre-flight: kiểm tra công cụ

Mục tiêu: xác nhận môi trường trước khi tạo project để tránh debug nhầm code trong khi lỗi nằm ở toolchain.

```
java -version
mvn -v
git --version

# Kiểm tra SQL Server bằng SSMS/Azure Data Studio hoặc sqlcmd nếu có
# Kiểm tra port backend dự kiến: 8080
```

| Kiểm tra | PASS | FAIL xử lý |
| --- | --- | --- |
| Java | JDK 21 được nhận | Sửa JAVA_HOME/PATH |
| Maven | mvn -v chạy | Cài Maven hoặc dùng mvnw |
| SQL Server | Connect instance được | Kiểm tra service/instance/port/login |
| Port 8080 | Không bị app khác chiếm | Đóng process hoặc đổi server.port |

### 6.1.2. Bước 1 – Tạo database lab và seed tối thiểu

Mục tiêu: có database rõ ràng để backend kết nối. Nếu dùng database Assignment 02 có sẵn, phải thống nhất tên database trong script và `application.properties`.

```
IF DB_ID('FUNewsManagementSBA301') IS NULL
    CREATE DATABASE FUNewsManagementSBA301;
GO
USE FUNewsManagementSBA301;
GO

IF OBJECT_ID('dbo.Category','U') IS NULL
BEGIN
    CREATE TABLE dbo.Category(
        CategoryID SMALLINT IDENTITY(1,1) PRIMARY KEY,
        CategoryName NVARCHAR(100) NOT NULL,
        CategoryDesciption NVARCHAR(250) NOT NULL,
        ParentCategoryID SMALLINT NULL,
        IsActive BIT NULL
    );
END;
GO

IF NOT EXISTS (SELECT 1 FROM dbo.Category)
BEGIN
    INSERT dbo.Category(CategoryName,CategoryDesciption,ParentCategoryID,IsActive)
    VALUES
      (N'Academic news',N'Academic announcements',NULL,1),
      (N'Student Affairs',N'Student activities and events',NULL,1),
      (N'Campus Safety',N'Campus safety information',NULL,1);
END;
GO
SELECT * FROM dbo.Category;
```

Kiểm chứng: query cuối phải trả ít nhất 3 rows. Evidence: screenshot/query result hoặc SQL output.

### 6.1.3. Bước 2 – Tạo Spring Boot project

11. IntelliJ IDEA → New Project → Spring Initializr.
12. Language: Java; Build: Maven; JDK: 21; Packaging: Jar.
13. Artifact gợi ý: demo-slot22-backend.
14. Dependencies: Spring Web, Spring Data JPA, Validation, MS SQL Server Driver, Spring Boot Starter Test.

Nếu tạo project bằng start.spring.io, giải nén rồi mở bằng IntelliJ. Từ CMD tại thư mục project có thể dùng `idea .` nếu IntelliJ command-line launcher đã có trong PATH.

```
cd D:\SBA301\demo-slot22-backend
idea .
```

Kiểm chứng: Maven import hoàn tất, không còn dependency màu đỏ. Chạy compile trước khi viết code:

```
mvnw.cmd -q -DskipTests compile
# hoặc nếu Maven đã có PATH
mvn -q -DskipTests compile
```

PASS: BUILD SUCCESS. FAIL: xử lý JDK/dependency trước khi sang bước tiếp theo.

### 6.1.4. Bước 3 – Kiểm tra pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```
mvnw.cmd dependency:tree
```

Kiểm chứng: dependency tree có Spring Web, Data JPA, Validation và mssql-jdbc; không có version conflict nghiêm trọng.

### 6.1.5. Bước 4 – Tạo cấu trúc package

```
src/main/java/com/fpt/sba301/slot22/
├─ Slot22Application.java
├─ controller/
│  └─ CategoryController.java
├─ dto/
│  ├─ CategoryRequest.java
│  └─ CategoryResponse.java
├─ entity/
│  └─ Category.java
├─ exception/
│  ├─ ResourceNotFoundException.java
│  ├─ BusinessConflictException.java
│  └─ ApiExceptionHandler.java
├─ repository/
│  └─ CategoryRepository.java
└─ service/
   ├─ CategoryService.java
   └─ CategoryServiceImpl.java

src/main/resources/
└─ application.properties
```

Quy tắc: main application class đặt ở package root để component scan mặc định bao phủ sub-packages. Không cần lạm dụng `@ComponentScan`, `@EnableJpaRepositories`, `@EntityScan` nếu package structure hợp lý.

### 6.1.6. Bước 5 – Cấu hình application.properties

```
spring.application.name=demo-slot22-backend
server.port=8080

spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=FUNewsManagementSBA301;encrypt=true;trustSe
rverCertificate=true
spring.datasource.username=sa
spring.datasource.password=YOUR_PASSWORD
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Nếu schema chưa được tạo trước và mục đích lab là cho Hibernate tạo bảng, có thể tạm dùng `ddl-auto=update`; nhưng với script đã có sẵn, `validate` giúp phát hiện mismatch mapping/schema thay vì âm thầm sửa DB.

Kiểm chứng cấu hình bằng cách chạy application ngay cả khi chưa có controller:

```
mvnw.cmd spring-boot:run
```

PASS: log có “Started ...Application”, DataSource kết nối được, không có mapping/schema validation error. FAIL: ưu tiên đọc root cause đầu tiên của SQLException/Hibernate exception.

### 6.1.7. Bước 6 – Tạo Entity Category

```
package com.fpt.sba301.slot22.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "Category")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "CategoryID")
    private Short categoryId;

    @Column(name = "CategoryName", nullable = false, length = 100)
    private String categoryName;

    // Tên cột thật trong database Assignment 02 đang là CategoryDesciption.
    @Column(name = "CategoryDesciption", nullable = false, length = 250)
    private String categoryDescription;

    @Column(name = "ParentCategoryID")
    private Short parentCategoryId;

    @Column(name = "IsActive")
    private Boolean active;

    public Category() {}
    // Generate getters/setters bằng IDE.
}
```

Thao tác IntelliJ: đặt cursor trong class → Code → Generate → Getter and Setter → chọn fields. Không viết tay hàng loạt nếu IDE có thể sinh chính xác.

Kiểm chứng: restart application với `ddl-auto=validate`. PASS nếu Hibernate không báo missing table/column/type mismatch.

### 6.1.8. Bước 7 – Tạo Repository

```
package com.fpt.sba301.slot22.repository;

import com.fpt.sba301.slot22.entity.Category;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CategoryRepository extends JpaRepository<Category, Short> {
    boolean existsByCategoryNameIgnoreCase(String categoryName);
    Page<Category> findByCategoryNameContainingIgnoreCase(String keyword, Pageable pageable);
}
```

Kiểm chứng startup: nếu derived-query property sai, application thường fail khi Spring Data tạo repository query. PASS khi app start và repository bean được tạo.

### 6.1.9. Bước 8 – Tạo Request/Response DTO

```
package com.fpt.sba301.slot22.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record CategoryRequest(
    @NotBlank(message = "Category name is required")
    @Size(max = 100)
    String categoryName,

    @NotBlank(message = "Description is required")
    @Size(max = 250)
    String categoryDescription,

    Short parentCategoryId,
    Boolean active
) {}
```

```
package com.fpt.sba301.slot22.dto;

public record CategoryResponse(
    Short categoryId,
    String categoryName,
    String categoryDescription,
    Short parentCategoryId,
    Boolean active
) {}
```

Kiểm chứng compile: `mvnw.cmd -q -DskipTests compile`. Nếu `jakarta.validation` không resolve, kiểm tra starter-validation.

### 6.1.10. Bước 9 – Tạo Service contract

```
public interface CategoryService {
    Page<CategoryResponse> findAll(String keyword, Pageable pageable);
    CategoryResponse getById(short id);
    CategoryResponse create(CategoryRequest request);
    CategoryResponse update(short id, CategoryRequest request);
    void delete(short id);
}
```

Mục tiêu: Controller chỉ biết use case contract, không biết query chi tiết.

### 6.1.11. Bước 10 – Tạo exception classes

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) { super(message); }
}

public class BusinessConflictException extends RuntimeException {
    public BusinessConflictException(String message) { super(message); }
}
```

### 6.1.12. Bước 11 – Implement Service

```
@Service
public class CategoryServiceImpl implements CategoryService {
    private final CategoryRepository repository;

    public CategoryServiceImpl(CategoryRepository repository) {
        this.repository = repository;
    }

    @Override
    @Transactional(readOnly = true)
    public Page<CategoryResponse> findAll(String keyword, Pageable pageable) {
        Page<Category> page = (keyword == null || keyword.isBlank())
                ? repository.findAll(pageable)
                : repository.findByCategoryNameContainingIgnoreCase(keyword, pageable);
        return page.map(this::toResponse);
    }

    @Override
    @Transactional(readOnly = true)
    public CategoryResponse getById(short id) {
        return toResponse(findEntity(id));
    }

    @Override
    @Transactional
    public CategoryResponse create(CategoryRequest request) {
        if (repository.existsByCategoryNameIgnoreCase(request.categoryName())) {
            throw new BusinessConflictException("Category name already exists");
        }
        Category c = new Category();
        apply(c, request);
        return toResponse(repository.save(c));
    }

    @Override
    @Transactional
    public CategoryResponse update(short id, CategoryRequest request) {
        Category c = findEntity(id);
        apply(c, request);
        return toResponse(repository.save(c));
    }

    @Override
    @Transactional
    public void delete(short id) {
        Category c = findEntity(id);
        repository.delete(c);
    }

    private Category findEntity(short id) {
        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Category " + id + " not found"));
    }

    private void apply(Category c, CategoryRequest r) {
        c.setCategoryName(r.categoryName().trim());
        c.setCategoryDescription(r.categoryDescription().trim());
        c.setParentCategoryId(r.parentCategoryId());
        c.setActive(r.active() == null ? Boolean.TRUE : r.active());
    }
```

```
    private CategoryResponse toResponse(Category c) {
        return new CategoryResponse(c.getCategoryId(), c.getCategoryName(),
            c.getCategoryDescription(), c.getParentCategoryId(), c.getActive());
    }
}
```

Lưu ý bài tập tiếp theo sẽ bổ sung rule “không xóa Category đang được NewsArticle tham chiếu”. Ở checkpoint Category-only chưa có NewsArticleRepository nên chưa giả lập rule đó.

Kiểm chứng: compile + unit reasoning. Đặt breakpoint tại `create()` và xác nhận repository được inject qua constructor.

### 6.1.13. Bước 12 – Tạo REST Controller

```
@RestController
@RequestMapping("/api/categories")
public class CategoryController {
    private final CategoryService service;

    public CategoryController(CategoryService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<Page<CategoryResponse>> getAll(
            @RequestParam(required = false) String keyword,
            @PageableDefault(size = 10, sort = "categoryName") Pageable pageable) {
        return ResponseEntity.ok(service.findAll(keyword, pageable));
    }

    @GetMapping("/{id}")
    public ResponseEntity<CategoryResponse> getById(@PathVariable short id) {
        return ResponseEntity.ok(service.getById(id));
    }

    @PostMapping
    public ResponseEntity<CategoryResponse> create(@Valid @RequestBody CategoryRequest request) {
        CategoryResponse created = service.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<CategoryResponse> update(
            @PathVariable short id,
            @Valid @RequestBody CategoryRequest request) {
        return ResponseEntity.ok(service.update(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable short id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Kiểm chứng routing bằng startup log hoặc gọi endpoint GET. Nếu nhận 404, kiểm tra `/api/categories`, port và class có `@RestController`.

### 6.1.14. Bước 13 – Global exception handling

```
public record ApiError(String code, String message, Instant timestamp) {}

@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError("NOT_FOUND", ex.getMessage(), Instant.now()));
    }

    @ExceptionHandler(BusinessConflictException.class)
    ResponseEntity<ApiError> handleConflict(BusinessConflictException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(new ApiError("CONFLICT", ex.getMessage(), Instant.now()));
    }
}
```

Bean Validation errors có thể được Spring xử lý mặc định ở course baseline; bài nâng cao yêu cầu chuẩn hóa luôn field-level validation error.

### 6.1.15. Bước 14 – Chạy backend

```
# Windows CMD
mvnw.cmd clean test
mvnw.cmd spring-boot:run

# Hoặc Maven global
mvn clean test
mvn spring-boot:run
```

PASS: application start ở port 8080; SQL log có query khi gọi API. Evidence: startup log có “Started ...Application”.

### 6.1.16. Bước 15 – Kiểm thử GET list + pagination

```
GET http://localhost:8080/api/categories?page=0&size=10&sort=categoryName,asc
```

Expected: HTTP 200; body là Page JSON có `content`, paging metadata và seed categories. Sau đó chạy SQL `SELECT * FROM dbo.Category ORDER BY CategoryID;` để đối chiếu dữ liệu.

### 6.1.17. Bước 16 – Kiểm thử GET detail và negative case

```
GET http://localhost:8080/api/categories/1
GET http://localhost:8080/api/categories/32767
```

| Request | Expected |
|---|---|
| Existing ID | 200 + category JSON |
| Missing ID | 404 + ApiError NOT_FOUND |

### 6.1.18. Bước 17 – Kiểm thử POST Create

```
POST http://localhost:8080/api/categories
Content-Type: application/json

{
  "categoryName": "Technology News",
  "categoryDescription": "Technology and software engineering news",
  "parentCategoryId": null,
  "active": true
}
```

Expected: 201 Created và có `categoryId`. Kiểm chứng database:

```
SELECT TOP 10 * FROM dbo.Category ORDER BY CategoryID DESC;
```

PASS chỉ khi API trả 201 VÀ row tồn tại trong SQL Server.

### 6.1.19. Bước 18 – Kiểm thử validation và duplicate

```
POST /api/categories
{
  "categoryName": "",
  "categoryDescription": "",
  "active": true
}
```

Expected: 400; database không có row mới. Sau đó POST lại tên đã tồn tại; expected 409 nếu duplicate rule được bật.

### 6.1.20. Bước 19 – Kiểm thử PUT Update

```
PUT http://localhost:8080/api/categories/1
Content-Type: application/json

{
  "categoryName": "Academic & Research News",
  "categoryDescription": "Academic and research announcements",
  "parentCategoryId": null,
  "active": true
}
```

Expected: 200. Verify bằng GET cùng id và SQL query. Đặt breakpoint Service.update để quan sát find → mutate → save/dirty checking.

### 6.1.21. Bước 20 – Kiểm thử DELETE

```
DELETE http://localhost:8080/api/categories/{id}
```

Với category test không bị tham chiếu: expected 204; GET lại expected 404; SQL row không còn nếu dùng hard delete. Không dùng category seed đang cần cho bước mở rộng.

### 6.1.22. Bước 21 – Search/filter

```
GET http://localhost:8080/api/categories?keyword=academic&page=0&size=5
```

Expected: 200 và `content` chỉ chứa category name match theo derived query ignore-case.

### 6.1.23. Bước 22 – Trace một CREATE end-to-end bằng debugger

15. Đặt breakpoint tại CategoryController.create().
16. Đặt breakpoint tại CategoryServiceImpl.create().
17. Đặt breakpoint trước repository.save().
18. Gửi POST từ Postman.
19. Quan sát request DTO, service rule, entity trước save và entity sau save có generated ID.
20. Bật SQL log để nhìn INSERT.
21. Query DB sau khi request hoàn tất.

```
POST JSON
 -> CategoryController.create
 -> CategoryServiceImpl.create
 -> CategoryRepository.save (proxy)
 -> SimpleJpaRepository / EntityManager (framework)
 -> Hibernate INSERT
 -> SQL Server commit
 <- generated ID
 <- 201 JSON
```

Evidence bắt buộc: screenshot breakpoint hoặc debug note + SQL log + Postman response + DB row.

## 6.2. Bài tập chủ đề nâng cao – Bổ sung business delete rule bằng NewsArticle

Mục tiêu: chuyển từ CRUD kỹ thuật sang CRUD có business constraint đúng Assignment 02: không cho xóa Category nếu đã được NewsArticle sử dụng.

### 6.2.1. Bước A – Map NewsArticle tối thiểu

```
@Entity
@Table(name = "NewsArticle")
public class NewsArticle {
    @Id
    @Column(name = "NewsArticleID", length = 20)
    private String newsArticleId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "CategoryID")
    private Category category;

    // Chỉ map thêm fields cần cho bài tập hoặc map đầy đủ theo schema Assignment.
}
```

### 6.2.2. Bước B – Repository kiểm tra tham chiếu

```
public interface NewsArticleRepository extends JpaRepository<NewsArticle, String> {
    boolean existsByCategory_CategoryId(Short categoryId);
}
```

### 6.2.3. Bước C – Cập nhật Service.delete

```
@Transactional
public void delete(short id) {
    Category c = findEntity(id);
    if (newsArticleRepository.existsByCategory_CategoryId(id)) {
        throw new BusinessConflictException(
            "Category is used by one or more news articles");
    }
    repository.delete(c);
}
```

### 6.2.4. Bước D – Kiểm chứng 409

Chọn một CategoryID đang xuất hiện trong NewsArticle. Gửi DELETE. Expected: 409; row Category vẫn tồn tại; NewsArticle không bị thay đổi. Đây là bằng chứng business rule được xử lý trước database mutation.

## 6.3. Break-It → Diagnose → Fix workshop

| Break | Triệu chứng | Cách chẩn đoán | Fix |
|---|---|---|---|
| Sai databaseName | Startup fail/SQL exception | Đọc datasource URL + DB name | Sửa application.properties |
| Đổi @Column thành CategoryDescription trong khi DB là CategoryDesciption | Schema validate fail/invalid column | So Entity với INFORMATION_SCHEMA | Map đúng tên hoặc migrate schema |
| Derived query sai property | Startup repository creation fail | Đọc property trong Entity | Đổi method name |
| Bỏ @RequestBody | 400/binding không như mong đợi | Controller signature | Thêm @RequestBody |
| Gửi text/plain | 415 | Postman headers | Content-Type: application/json |
| Controller gọi repository trực tiếp | Vẫn chạy nhưng architecture fail | Code review boundary | Chuyển qua Service |
| DELETE category đang dùng | 409 mong đợi; nếu bị xóa là | Check service + DB | exists check + conflict |

| | rule sai | | |
|---|---|---|---|
| Return 200 cho missing id | Contract sai | Negative test | Map not found -> 404 |

## 6.4. Automated test – bài tập củng cố

### 6.4.1. Controller test

```
@WebMvcTest(CategoryController.class)
class CategoryControllerTest {
    @Autowired MockMvc mvc;
    @MockBean CategoryService service;

    @Test
    void getMissing_returns404() throws Exception {
        when(service.getById((short)99))
            .thenThrow(new ResourceNotFoundException("Category 99 not found"));
        mvc.perform(get("/api/categories/99"))
            .andExpect(status().isNotFound());
    }
}
```

### 6.4.2. Service test ideas

- create duplicate name -> BusinessConflictException.
- get missing id -> ResourceNotFoundException.
- delete referenced category -> BusinessConflictException.
- update existing -> repository.save được gọi và field thay đổi đúng.

Lệnh chạy:

```
mvnw.cmd test
# Chạy một test class
mvnw.cmd -Dtest=CategoryControllerTest test
```

## 6.5. Bài tập tổng hợp hệ thống – Backend readiness before React Part B

Sinh viên mở rộng từ Category sang NewsArticle backend nhưng vẫn khóa ở backend. Không xây React ở Slot 22.

| Gate | Yêu cầu | Evidence |
|---|---|---|
| G1 Build | clean test pass | Maven output |
| G2 DB | Đúng database/schema/seed | SQL query |
| G3 Entity | Category + NewsArticle mapping đúng | startup validate |
| G4 Repository | CRUD + search/query cần thiết | repository code/test |
| G5 Service | Business rules + transaction | source + unit test |
| G6 Controller | GET/POST/PUT/DELETE/search | Postman collection |
| G7 HTTP | 200/201/204/400/404/409 đúng | test matrix |
| G8 Persistence | Create/update/delete verify DB | before/after SQL |
| G9 Trace | Trace ít nhất 1 POST hoặc PUT | debug/sequence note |
| G10 Explain | Giải thích Controller-Service-Repository-JPA-SQL | human verification |

### 6.5.1. Test matrix bắt buộc

| ID | Request | Expected HTTP | DB verify |
|---|---|---|---|
| T01 | GET categories | 200 | no mutation |
| T02 | GET existing id | 200 | same row |
| T03 | GET missing id | 404 | no mutation |
| T04 | POST valid | 201 | new row |
| T05 | POST invalid | 400 | no new row |
| T06 | POST duplicate | 409 | no duplicate |
| T07 | PUT existing | 200 | fields updated |

| | | | |
|---|---|---|---|
| T08 | PUT missing | 404 | no mutation |
| T09 | DELETE free category | 204 | row removed |
| T10 | DELETE referenced category | 409 | row retained |
| T11 | Search keyword | 200 | filtered rows |
| T12 | Pagination | 200 | bounded page |

### 6.5.2. Submission evidence

- Source code repository / ZIP.
- README: prerequisites, database name, run commands, endpoint table.
- SQL initialization/seed script hoặc hướng dẫn schema strategy.
- Postman collection.
- Test matrix result.
- Ít nhất 1 execution trace.
- Ít nhất 1 Break-It → Diagnose → Fix note.
- SQL screenshots/query outputs cho create/update/delete post-condition.

## VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

| Thuật ngữ | Định nghĩa ngắn | Ví dụ Slot 22 |
|---|---|---|
| CRUD | Create, Read, Update, Delete | Category CRUD |
| REST resource | Đối tượng được biểu diễn qua URI | /api/categories |
| Entity | Object ánh xạ persistence | Category |
| Repository | Abstraction truy cập data | CategoryRepository |
| JpaRepository | Repository interface có CRUD/JPA operations | JpaRepository<Category,Short> |
| Service | Application/business use case layer | CategoryService |
| Transaction | Đơn vị công việc atomic trong DB | create/update/delete |
| Managed entity | Entity đang được persistence context quản lý | loaded Category trong transaction |
| Dirty checking | Hibernate phát hiện entity thay đổi | UPDATE khi transaction flush |
| Persist | Đưa entity mới vào persistence context | INSERT |
| Merge | Copy state entity detached vào managed instance | update concept |
| DTO | API-facing data shape | CategoryRequest |
| Bean Validation | Constraint validation | @NotBlank |
| Business rule | Quy tắc nghiệp vụ | cannot delete referenced category |
| Foreign key | Ràng buộc tham chiếu | NewsArticle.CategoryID |
| 409 Conflict | Request hợp lệ cú pháp nhưng conflict state/rule | delete category in use |
| ProblemDetail | Structured HTTP problem response | RFC 9457 direction |
| Pageable | Paging/sorting request abstraction | page,size,sort |
| Projection | Chọn subset fields cho query result | summary response |
| Optimistic locking | Phát hiện concurrent update bằng version | @Version awareness |

# VIII. RESEARCH BASE VÀ AI BASE LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

## 8.1. Quy tắc evidence

- AI không được tự bịa class/method/schema.
- Execution claim → source/debugger/log/SQL evidence.
- Framework/version claim → official Spring documentation/release notes.
- Phân biệt Course Baseline và Current Production Radar.
- Sinh viên lưu prompt + answer + evidence + kết luận cá nhân.
- Nhóm II, III và IV có thêm 10 prompt mở rộng/nhóm; sinh viên không bắt buộc chạy tất cả trong một buổi. Giảng viên có thể chọn 2–4 prompt/nhóm theo mục tiêu lớp, phần còn lại dùng cho tự học hoặc project mentoring.

## 8.2. Nhóm 1 – Phân tích workflow và quy luật thực thi

```
PROMPT 1
Đóng vai Senior Spring Boot Mentor. Phân tích source demo_slot22 mà tôi cung cấp. Chọn POST /api/categories và
trace từ HTTP request tới SQL Server rồi response quay về client. Không tự tạo method không có trong source. Với
mỗi bước ghi file/class/method/input/output/dependency/object tham gia. Tách rõ code của tôi và framework
concept. Tạo breakpoint/log plan để tôi kiểm chứng bằng debugger và Hibernate SQL.

PROMPT 2
So sánh lifecycle CREATE và UPDATE trong JPA project này. Giải thích entity new/managed/detached, persist/merge,
dirty checking, flush, transaction commit. Chỉ ra phần nào quan sát được bằng debugger/SQL log và phần nào chỉ
là conceptual framework.
```

## 8.3. Nhóm 2 – Prompt vấn đề nâng cao

```
PROMPT II-01
Phân tích delete Category khi có NewsArticle tham chiếu. So sánh: application-level exists check, FK constraint,
cascade delete, soft delete. Với mỗi lựa chọn nêu invariant, race condition có thể xảy ra, transaction/isolation
implication và test case cần có. Không đề xuất cascade chỉ vì code ngắn.

PROMPT II-02
Từ endpoint list Category, thiết kế paging/filter/sort/search. So sánh Page vs Slice vs List; derived query vs
Specification; projection vs full entity. Đưa decision table theo dataset size và UI need.
```

Bổ sung 10 prompt mở rộng – kỹ thuật và hệ sinh thái liên quan

```
PROMPT II-03
Phân tích cơ chế persistence context của JPA trong một request CRUD. Hãy dùng ví dụ CategoryService.update(...):
entity được load khi nào, trở thành managed khi nào, dirty checking phát hiện thay đổi ra sao, flush xảy ra ở đâu và
SQL UPDATE xuất hiện lúc nào. So sánh với trường hợp tạo object mới rồi gọi save(). Yêu cầu: vẽ state transition, chỉ
ra breakpoint/log nên đặt, và nêu 5 hiểu sai thường gặp của sinh viên.

PROMPT II-04
Phân tích transaction boundary cho các use case CRUD trong Spring Boot. So sánh @Transactional ở Controller, Service
và Repository; readOnly=true; propagation mặc định; rollback khi RuntimeException; và nguy cơ partial update nếu một
use case có nhiều bước. Dùng FUNewsManagement làm ví dụ và kết thúc bằng decision table: method nào cần transaction,
method nào không, bằng chứng nào cần kiểm chứng.

PROMPT II-05
Giải thích N+1 query trong JPA bằng quan hệ Category -> NewsArticle. Tạo ba kịch bản: GET category list không cần
articles, GET category detail cần articles, và report cần một số field của article. So sánh LAZY/EAGER, fetch join,
EntityGraph và projection. Không khuyên đổi tất cả sang EAGER. Đưa cách phát hiện bằng Hibernate SQL log và số lượng
query kỳ vọng.

PROMPT II-06
Thiết kế validation cho Category và NewsArticle theo ba lớp: syntactic/Bean Validation, business validation và
database constraint. Ví dụ: name không rỗng, headline giới hạn độ dài, category phải tồn tại, email account duy nhất.
Với mỗi rule chỉ rõ nơi đặt rule, exception/status HTTP mong đợi, test case positive/negative và vì sao không nên dồn
mọi validation vào Controller.

PROMPT II-07
Phân tích concurrency khi hai request cùng update một NewsArticle. Giải thích lost update, optimistic locking với
@Version, pessimistic locking và database isolation ở mức người học SBA301. Thiết kế một experiment bằng hai Postman
request hoặc hai test thread để tái hiện vấn đề. Nêu lúc nào optimistic locking đáng dùng và lúc nào lab nhỏ chưa
cần.
```

PROMPT II-08
So sánh PUT và PATCH cho update REST API. Dùng NewsArticle làm ví dụ để chỉ ra full replacement, partial update, idempotency, null semantics và validation. Đề xuất DTO cho PUT và PATCH, nêu rủi ro khi copy toàn bộ field từ request vào entity, và tạo bảng test case cho missing field, null field, immutable field và nonexistent id.

PROMPT II-09
Phân tích các chiến lược mapping DTO <-> Entity trong Spring Boot: mapping thủ công, constructor/factory, MapStruct và ModelMapper. Đặt trong bối cảnh Assignment 02 có Category, NewsArticle, Account, Tag. So sánh compile-time safety, boilerplate, hidden behavior, nested relationship và testability. Kết thúc bằng khuyến nghị course baseline vs project lớn.

PROMPT II-10
Giải thích pagination và sorting từ HTTP request tới SQL query. Trace GET /api/news-articles?page=0&size=20&sort=createdDate,desc từ @RequestParam/Pageable -> Service -> JpaRepository -> generated SQL. Phân biệt Page và Slice, count query, offset pagination và keyset pagination. Tạo checklist để sinh viên dùng SQL log chứng minh query thật sự được phân trang.

PROMPT II-11
Phân tích search/filter backend cho NewsArticle. So sánh derived query, @Query JPQL, native SQL, Query by Example, Specification/Criteria API. Xây decision tree dựa trên số lượng filter động, join relationship, maintainability và performance. Yêu cầu tạo một ví dụ filter keyword + categoryId + status và chỉ ra cách test repository độc lập.

PROMPT II-12
Giải thích exception flow trong Spring MVC: exception phát sinh ở Repository/Service -> Controller -> @RestControllerAdvice -> ProblemDetail/response JSON. Dùng ResourceNotFound, validation failure, duplicate/conflict và FK constraint làm ví dụ. Thiết kế standardized error contract gồm code, message, fieldErrors, timestamp, path, traceId và nêu field nào không nên lộ ra client.

## 8.4. Nhóm 3 – Prompt chuyên nghiệp trong dự án production

PROMPT III-01
Đóng vai backend reviewer. Review CRUD backend này theo production checklist: DTO boundary, validation, standardized errors, transaction, idempotency, optimistic locking, audit fields, schema migration, secrets, pagination, observability, tests. Với mỗi issue nêu severity, evidence trong source, risk và minimal refactor. Không biến project lab thành microservices nếu không cần.

PROMPT III-02
Thiết kế migration từ ddl-auto=update sang controlled database migration cho FUNewsManagement. Đề xuất Flyway/Liquibase workflow ở mức khái niệm: baseline, versioned migration, rollback/forward-fix, CI validation, production safety. Phân biệt điều nên học ngay và điều chỉ awareness.

Bổ sung 10 prompt mở rộng – kỹ thuật và hệ sinh thái liên quan

PROMPT III-03
Đóng vai Senior Backend Reviewer và review project CRUD theo Clean Architecture/Layered Architecture ở mức vừa đủ. Kiểm tra dependency direction, package-by-layer vs package-by-feature, service interface có thực sự cần hay không, repository leak, DTO boundary và transaction boundary. Không ép microservices. Đưa ra refactor roadmap 3 mức: lab-ready, team-project-ready, production-ready.

PROMPT III-04
Thiết kế chiến lược database migration cho SQL Server trong CI/CD. So sánh ddl-auto=create/update/validate/none với Flyway và Liquibase. Xây workflow dev -> test -> staging -> production gồm versioned migration, checksum, baseline, repeatable migration, forward-fix và backup. Chỉ rõ vì sao ddl-auto=update không phải production migration strategy.

PROMPT III-05
Thiết kế observability cho CRUD backend: structured logging, correlation/trace ID, metrics, health checks và distributed tracing awareness. Dùng một POST NewsArticle thất bại làm case study. Chỉ ra log nào nên có ở Controller/Service/DB boundary, dữ liệu nào không được log (password/token/PII), và cách dùng Spring Boot Actuator/OpenTelemetry ở mức khái niệm.

PROMPT III-06
Phân tích API contract governance cho một team nhiều frontend/backend developer. Đề xuất OpenAPI workflow, naming convention, status code convention, error schema, pagination schema, versioning/deprecation policy và backward compatibility. Tạo checklist pull request cho endpoint CRUD mới và chỉ ra những thay đổi nào là breaking change.

PROMPT III-07
Phân tích security boundary cho CRUD backend trước khi học sâu Spring Security. Liệt kê threats liên quan input validation, mass assignment/over-posting, IDOR/BOLA, SQL injection, secret management, CORS misconception và error information leakage. Với mỗi threat nêu mitigation ở application/framework/database layer và một negative test có thể chạy.

PROMPT III-08
Thiết kế audit trail cho NewsArticle: ai tạo, ai sửa, thời điểm, trạng thái trước/sau. So sánh JPA Auditing (@CreatedDate/@LastModifiedDate/@CreatedBy/@LastModifiedBy), audit table riêng và event log. Nêu trade-off về compliance, storage, queryability và tamper resistance. Đề xuất mức tối thiểu phù hợp Assignment 02.

PROMPT III-09
Phân tích soft delete vs hard delete cho Category, Account và NewsArticle. Xét referential integrity, legal/audit requirement, unique constraints, search/filter mặc định, restore, storage growth và accidental data exposure. Thiết kế API semantics và database columns cho soft delete, đồng thời nêu khi nào hard delete vẫn hợp lý.

PROMPT III-10
Thiết kế testing pyramid cho CRUD backend: unit Service test với Mockito, @DataJpaTest cho Repository, @WebMvcTest cho Controller và @SpringBootTest/Testcontainers cho integration. Chọn 12 test quan trọng nhất cho Category/NewsArticle và giải thích bug class nào mỗi tầng bắt được hoặc không bắt được.

PROMPT III-11
Phân tích performance baseline cho CRUD API trước khi tối ưu. Đề xuất cách đo latency p50/p95, throughput, DB query count, connection pool usage và payload size. Dùng endpoint list/search NewsArticle làm ví dụ. Phân biệt evidence-based optimization với premature optimization và đưa ra checklist profiling trước khi thêm cache/index.

PROMPT III-12
Đóng vai Database Architect. Review schema FUNewsManagement về PK/FK, index, nullability, data types, unique constraints và cascade rules. Đặc biệt xem Category self-reference, NewsArticle -> Category/Account và NewsTag join table. Đề xuất index cho search/filter phổ biến nhưng yêu cầu giải thích write-cost và chứng minh bằng execution plan thay vì đoán.

## 8.5. Nhóm 4 – Prompt xu hướng/cập nhật cộng đồng hiện tại

PROMPT IV-01
Tôi đang học Spring Boot 3.x + JDK 21. Hãy kiểm tra official docs hiện tại và tạo bảng Course Baseline vs Current Ecosystem cho Spring Boot, Spring Framework MVC validation, Spring Data JPA, ProblemDetail/RFC 9457, testing và database migration. Ghi version/ngày nguồn/URL/ngày truy cập. Phân loại MUST KNOW NOW / SHOULD EXPLORE / WATCH LATER. Không dùng blog thứ cấp làm nguồn duy nhất.

PROMPT IV-02
Tìm trong official Spring Data JPA docs hiện tại các feature liên quan trực tiếp CRUD backend: projection, Specification, auditing, locking, Query by Example, AOT. Với mỗi feature trả lời: vấn đề nó giải quyết, khi nào lab nhỏ chưa cần, một ví dụ tối thiểu, và cách kiểm chứng.

Bổ sung 10 prompt mở rộng – kỹ thuật và hệ sinh thái liên quan

PROMPT IV-03
Kiểm tra official Spring Boot và Spring Framework documentation hiện tại về REST error handling và ProblemDetail. So sánh course code ResponseEntity custom error với framework support hiện hành. Ghi version, nguồn chính thức, ngày truy cập, API liên quan và phân loại: MUST KNOW NOW / SHOULD EXPLORE / WATCH LATER.

PROMPT IV-04
Khảo sát official Spring Data JPA documentation hiện tại về repository features: projections, Specifications, Query by Example, auditing, locking và pagination. Tạo bảng feature -> vấn đề giải quyết -> maturity -> khi nào dùng -> khi nào không nên dùng trong lab. Mọi claim version phải có nguồn chính thức.

PROMPT IV-05
Tìm hiểu xu hướng Java/Jakarta Persistence hiện tại liên quan CRUD backend. Kiểm tra official Jakarta Persistence specification và Hibernate documentation để xác định các thay đổi đáng chú ý về API, annotations, UUID/ID generation, query, schema generation hoặc compatibility với Spring Boot. Tách rõ kiến thức nền bền vững và nội dung migration awareness.

PROMPT IV-06
Khảo sát cách cộng đồng Spring hiện nay quản lý database schema trong project production. Dùng official docs của Flyway/Liquibase/Spring Boot làm nguồn chính. So sánh migration-on-startup, migration trong CI/CD, validate-only và zero-downtime migration patterns. Không lấy số liệu phổ biến từ blog nếu không có bằng chứng.

PROMPT IV-07
Khảo sát xu hướng REST client/API testing hiện tại cho Spring Boot backend: MockMvc, WebTestClient, REST Assured, Testcontainers và contract testing. Dùng official docs để tạo learning radar cho sinh viên SBA301: công cụ nào học ngay, công cụ nào nên thử trong project, công cụ nào để sau.

PROMPT IV-08
Tìm hiểu current best practices về Spring Boot configuration/secrets: application.properties/yaml, profiles, environment variables, configuration properties, secret manager awareness và 12-factor config. Dùng official Spring docs. Tạo migration plan từ password hard-code trong lab sang cấu hình an toàn hơn mà vẫn giữ setup sinh viên đơn giản.

PROMPT IV-09
Khảo sát xu hướng API documentation hiện tại trong Spring ecosystem: OpenAPI/Swagger ecosystem, generated docs, contract-first vs code-first. Chỉ dùng official Spring/standard/tool docs có uy tín. So sánh lợi ích với Postman collection và đề xuất cách sinh viên có thể thêm API documentation vào CRUD backend mà không làm lệch mục tiêu môn học.

PROMPT IV-10
Tìm hiểu xu hướng observability hiện tại trong Spring Boot: Micrometer, Actuator, OpenTelemetry, logs/metrics/traces. Kiểm tra official docs và lập bản đồ từ System.out.println của người mới tới structured observability production. Đưa một mini-experiment đo request latency và trace một lỗi 500.

PROMPT IV-11
Khảo sát current guidance về virtual threads/Java concurrency trong Spring Boot đối với CRUD web applications. Chỉ dùng nguồn chính thức Java/Spring. Giải thích khi nào virtual threads có thể liên quan I/O-bound request, giới hạn với database connection pool, và vì sao không nên đưa vào lab chỉ vì là công nghệ mới.

PROMPT IV-12
Tạo '2026 Backend CRUD Technology Radar' cho sinh viên SBA301 bằng nguồn chính thức: Spring Boot, Spring Framework, Spring Data JPA, Hibernate/Jakarta Persistence, SQL Server/JDBC, validation, testing, migration, observability và API documentation. Với mỗi mục ghi current stable direction, course baseline, khoảng cách cần biết, migration risk và một bài thử nhỏ để tự kiểm chứng.

## 8.6. AI Verification Worksheet

| Claim AI | Nguồn kiểm chứng | Thử nghiệm | Kết luận SV |
|---|---|---|---|
| save() create/update thế nào | Source + Spring Data docs + SQL log | breakpoint + SQL | |
| Delete rule đủ an toàn? | Service + FK + concurrency reasoning | 2 negative tests | |
| ProblemDetail phù hợp? | Spring official docs | MockMvc error response | |
| Projection giúp gì? | Spring Data official docs | compare query/result | |

# IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

## 9.1. Self-check kiến thức

- ☐ Tôi giải thích được vì sao CRUD database không hoàn toàn tương đương REST semantics không?
- ☐ Tôi phân biệt trách nhiệm Controller/Service/Repository bằng ví dụ cụ thể không?
- ☐ Tôi giải thích save(), persist/merge và dirty checking ở mức conceptual không?
- ☐ Tôi biết tại sao update phải kiểm tra resource tồn tại trước không?
- ☐ Tôi biết khi nào dùng 400, 404, 409 và 500 không?
- ☐ Tôi phân biệt Bean Validation, business validation và DB constraint không?
- ☐ Tôi giải thích được transaction boundary đặt ở đâu và vì sao không?
- ☐ Tôi biết vì sao delete có thể là business use case phức tạp không?
- ☐ Tôi biết vì sao production không nên findAll() vô hạn không?
- ☐ Tôi biết DTO giải quyết coupling nào giữa API và Entity không?

## 9.2. Self-check thực hành/evidence

- ☐ Backend build/start sạch.
- ☐ Datasource trỏ đúng database.
- ☐ GET list/detail pass và missing ID = 404.
- ☐ POST valid = 201; invalid = 400; duplicate = 409 nếu áp dụng.
- ☐ PUT existing pass; missing = 404.
- ☐ DELETE unused = 204; referenced = 409.
- ☐ Có SQL evidence cho create/update/delete.
- ☐ Có execution trace một request.
- ☐ Có ít nhất một Break-It → Diagnose → Fix note.
- ☐ Có README/API contract để frontend team dùng ở Part B.

## 9.3. Exit ticket 5 phút

| Câu hỏi | Sinh viên trả lời ngắn |
|---|---|
| 1. Một lỗi backend hôm nay tôi đã tự chẩn đoán là gì? | |
| 2. Một business rule tôi đã đặt đúng ở Service là gì? | |
| 3. Một evidence chứng minh API đúng là gì? | |
| 4. Một điểm lab khác production là gì? | |
| 5. Một câu hỏi tôi sẽ kiểm chứng tiếp ở Part B là gì? | |

**EXIT GATE:** READY FOR PART B khi backend contract ổn định, CRUD positive/negative pass, DB state được verify, và frontend có thể gọi API mà không cần biết persistence chi tiết.

# NGUỒN VÀ TRACEABILITY

- Course source: Chapter 15 – Implement CRUD Application with RESTful Web Service and ReactJS (Part 1: Spring Boot API project).
- Assessment source: Assignment 02 – FUNewsManagementSystem; 3-layer Spring REST API, SQL Server, validation, CRUD/Search business rules.
- Database source: Assignment 02_FUNewsManagementSBA301.sql – Category, NewsArticle, NewsTag, SystemAccount, Tag and foreign keys/seed data.
- Template source: SBA301 Slot 18 lecture – evidence-based flow Prepare → Understand → Trace → Implement → Break → Diagnose → Verify → Transfer.
- Production update reference (accessed 2026-10-12): Spring Data JPA official current docs (4.1.1); Spring Framework MVC Validation official docs (7.0.9); Spring HTTP error responses/ProblemDetail official docs.

Tài liệu này chủ ý giữ Spring Boot 3.x + JDK 21 làm course baseline. Những phần “production radar” là mở rộng để sinh viên hiểu hướng phát triển, không phải yêu cầu nâng version giữa kỳ.